

Name: _____

Collaborators: _____

THAYER SCHOOL OF ENGINEERING • DARTMOUTH

ENGS 28, W26

LAB 9 — FINAL PROJECT - BLUETOOTH

Due: Tuesday March 10, 2026 2:15 PM

Contents

1 Overview	3
1.1 Place in Course	3
1.2 Learning Objectives	3
1.3 Equipment	3
2 Design Challenge	3
2.1 The Goal	3
2.2 Project Requirements	4
3 Prelab	4
4 Procedure	5
4.1 USART1	5
4.2 UART mode	6
4.3 CMD mode	6
4.4 Switching modes	7
4.5 Build your system	8
4.6 Demo	8

List of Figures

List of Tables

1 Alternate Function Mappings	5
---	---

Deliverables

1 AF Settings for USART1 (2 pts)	4
2 Paper Design (5 pts)	4
3 USART1 driver (5 pts)	5
4 CMD Mode (3 pts)	7

5	BLE startup (5 points)	7
6	Demo! (5 pts)	8
7	Reflection (3 pts)	9

1 Overview

1.1 Place in Course

This is the culminating lab for this class. In it, you and a partner will bring together much of what you have learned in order to build and demonstrate a wireless system, whereby your Nucleo will send and receive information over Bluetooth to your phone or computer. The remote device (eg the phone) will be able to respond to and control your system over the same bluetooth connection.

1.2 Learning Objectives

- Learn to use a BLE¹ module (specifically the Adafruit Bluefruit LE UART)
- Connect your Nucleo to a remote device over BLE.
- Design a set of commands and responses you will exchange with the remote device.
- Begin to understand the application of de-centralized computing.

1.3 Equipment

- 1 Nucleo board connected to an Adafruit Bluefruit LE UART.
- Sensors of your choice
- Actuators of your choice
- At least one phone running [Adafruit Bluetooth application](#).
- (optional) a phone or tablet running the [Nordic Semiconductor BlueTooth App](#)
- (optional) a specialized app (provided by you) running on your phone, tablet, or laptop designed to communicate with you device.

2 Design Challenge

You will complete this lab with a partner.

This lab is intended to give you a great deal of freedom to choose exactly what you wish to implement.

2.1 The Goal

The overall goal is to build a system, which can reliably communicate with a Bluetooth partner. The system should have local control, but also exchange enough information with the Bluetooth partner that the Bluetooth partner can also effect the same control.

As a simple example, imagine you have a system with a push button to toggle a light. The system could reflect the current state of the light to the Bluetooth device. The Bluetooth device could also be used to turn on or turn off the light. This particular example is too simple to suffice as this project, but gives you an idea of what you might consider.

¹BLE stands for Bluetooth Low Energy. It uses the same radio as regular Bluetooth, but is otherwise quite different

You should also feel free (even encouraged) to be creative and use any of the components in your kit to construct something completely new!

2.2 Project Requirements

The requirements are listed here for reference and as a guide while you look through the documentation.

Your final system must include all of the following.

1. An embedded system of your design. The design must include:
 - At least 2 sensors.
 - At least 2 actuators.
2. A unique name which will allow you to easily identify your system in the sea of other projects
3. A “language” which you will use to interact with your system from your phone. The language must
 - include at least one command sent from the phone to the system with an observable result
 - include at least one piece of information which is updated periodically and can be seen on the phone.

3 Prelab

Read about the device we are using on the [Adafruit](#) website.

Pay particular attention to the discussion of the pinout on page 2. You can ignore the wiring diagrams because they target the old Arduino, not our Nucleo board.

Read this entire handout, then design on paper the system you will build. Note that the Bluefruit device requires the use of USART1. It also requires the use of hardware based flow control. Read the details of USART1 in the reference manual to understand how to enable this feature. Once you understand how to enable it, examine the alternate function mappings in the product data sheet. You will need to decide which physical pins you will use for the 4 USART1 signals.

Your paper design must include

1. A schematic of your system, including all components and connections.
2. A listing of all pins (Note the Arduino name and the Port/Pin) be sure to indicate the configuration also.
3. A software plan, including pseudo-code for your main loop and a list of any peripherals and/or interrupts you are planning to use.

Deliverable 1: AF Settings for USART1 (2 pts)

Fill in Table 1 below with the pins you intend to use.

Deliverable 2: Paper Design (5 pts)

USART Function	Port/Pin	Alt Func	Bluefruit Pin
USART1_TX			RXI
USART1_RX			
USART1_CTS			
USART1_RTS			CTS

Table 1: Alternate Function Mappings

Submit the three items listed above.

4 Procedure

This project has a number of significant parts, and you will do well to approach the Design as a series of small incremental steps. The following deliverables should lead you to a functioning system fairly quickly.

4.1 USART1

Your first task is to write a driver for USART1.

Design a driver for USART1 similar to that provided for USART2. You must enable flow control (RTS/CTS). You must provide an additional function for both USARTs that allows you to check to see if there is data available without blocking.

When you initialize USART1, you must enable flow control in order to make use of the Bluefruit device. Construct a driver similar to the one that we have given you already to make use of this new connection.

In order to test that your UART works properly, you will write a small program which connects USART2 (CoolTerm) to USART1. Any characters typed into CoolTerm will appear in the RX register for USART2. Copying them to the TX register for USART1 will send them along to whatever is connected to USART1. Making the corresponding connection in the other direction will ensure smooth bi-directional communication.

You can use your AD2 to observe the pins on USART1 in order to confirm that it is working properly. You should also use your AD2 to send a character to CoolTerm - look under the **Protocol** tab to see the interface you can use. You need to be a little careful (send 1 character at a time) because the AD2 *does not support* hardware flow control. *Remember* the AD2 TX signal must be connected to the RX of USART1. That way, anything *transmitted* from the AD2 will be *received* by USART1.

Using the two UARTs in this manner is known as *Passthrough*.

Important!

In the final system, you won't know which USART will transmit first, so you must devise a way to determine if there are any characters in the receive data register. Do not use the existing read function unless you *already know* there is a byte waiting, otherwise, you may block indefinitely.

The following deliverable assigns 4 points to a working driver and will award the remaining point for drivers implemented using interrupts. *It is strongly recommended that you get the system working without interrupts first, then modify it to use interrupts in place of polling*

Deliverable 3: USART1 driver (5 pts)

Submit your code and screenshots from Waveforms showing correct transmission from *CoolTerm* → *USART2_RX* → *USART1_TX*.

4.2 UART mode

Note: The [Adafruit docs](#) refer to this mode as “Data Mode”.

There’s no specific deliverable associated with this section, but you may end up spending a fair amount of time here debugging. Do not skip this step simply because there are no points attached. It is intended to make your final project bring up much more expedient.

Connect your BLE device to your Nucleo. Be sure to connect it as follows. The exact pins will depend on your alternate function choices.

Bluefruit Pin	Nucleo Pin
RXI	USART1_TX
TXO	USART1_RX
RTS	USART1_CTS
CTS	USART1_RTS
VIN	3V3
GND	GND

Slide the little switch on the bluetooth board to “UART”.

Now, with your Nucleo still running the passthrough program, open the Adafruit bluetooth app. There are likely to be a lot of devices listed. To make it easier to find yours, you can select the “Must have uart” filter. Once you find and connect to a device, open the UART tab.

Here, you can send and receive characters over bluetooth. This will be the primary interaction with your system. Test it out a bit, making sure that characters you type here appear on CoolTerm and vice versa.

If you have trouble finding your device, you may need to go to a space with fewer ES28 students trying to find their devices! Alternatively, take a look at the next section.

4.3 CMD mode

The bluetooth board has a second mode known as “Command Mode”. In this mode, you can send **AT** commands to the device over USART1. The exact format of the commands is described in the [Adafruit documentation](#).

Place your BLE device into command mode, by sliding the little switch toward the word CMD. In this mode, the device is expecting to read AT commands as described on the Adafruit website.

Experiment a bit with these commands. In particular, issue a reset (ATZ) by typing “ATZ” into CoolTerm.

A few of the most useful commands are

1. ATZ : (soft reset) - This command causes the device to restart the bluetooth stack, thereby broadcasting any changes to it’s setup.

2. `AT+FACTORYRESET` : The big hammer. You should issue this command as soon as your program starts up so you know exactly what state your device is in.
3. `AT+GAPDEVNAME` : This is how you give your device a recognizable name.

Issue a few of these commands from CoolTerm to see how they work (feel free to experiment with any other commands that look interesting!).

Once you have the hang of it, proceed.

Important!

You should still be running your “passthrough” code at this point. This will allow you to see if your commands were processed correctly (Bluetooth responds “OK”) or incorrectly (Bluetooth response “Error”)

Deliverable 4: CMD Mode (3 pts)

Submit a screenshot of your Coolterm window, connected to the device. Show the following commands and their responses.

- `AT+FACTORYRESET`
- `AT+GAPDEVNAME`
- `ATZ`

Check the docs for the exact syntax. Choose a unique name for your device.

4.4 Switching modes

Manually sliding the switch back-and-forth between UART and CMD mode is pretty time-consuming and somewhat beneath us at this point. Fortunately there is a way to do it through software. It requires the use of an additional GPIO pin, but is very easy to use.

Take a look at the schematic for the Bluetooth board. You can find it in the same place as all the other documentation. Look for the switch that controls the mode. You will note that in one position it connects to ground and in the other position it is pulled up to VDD.

If you connect a GPIO to the MOD pin on the board, you can control the mode by switching the pin high (CMD Mode) or low (UART mode).

Your final project should start in CMD mode and issue commands to do a hardware (factory) reset, followed by setting the device name, and finally issuing a soft reset to cause the device to start advertising the new name. It should then switch to UART mode and start processing commands.

The deliverable below ties together many of the last steps and provides a solid startup routine from which base your final code.

Deliverable 5: BLE startup (5 points)

For this deliverable, you will write a small program which uses a GPIO to put the device in CMD mode, performs a factory reset, assigns a device name, performs a soft reset and then returns to UART mode. Consider placing the initialization in its own file as a function you can call from any program. Submit the code for the startup function and a screenshot of the Adafruit app showing that you were able to connect to your device - the device name should be visible in the screenshot.

Important!

There are two tricky items here.

1. After sending a command, you should wait a few milliseconds (using `delay_ms` is fine). After this short delay, you should process the response. You don't necessarily need to do anything with the response, but at the very least, you should loop watching for available characters until the response is done. You can tell it is done when you haven't seen a new character in 10 or 20 milliseconds.
2. After extracting (and possibly ignoring) the response, you should wait an additional 200ms before sending another command.

4.5 Build your system

Now that you have bluetooth working, time to build your system and integrate the command processing into your main loop.

Your system should behave normally even if there is no other device connected.

When a bluetooth device connects, it should start receiving data from your System, and your system should start responding to commands from the Bluetooth device. Assume the Bluetooth device will come and go periodically, and make sure that your main loop does not depend on any particular state of the Bluetooth system.

4.6 Demo

On the last day of class each team will demonstrate their project.

Deliverable 6: Demo! (5 pts)

- ☐ a. Explain what your system does.
- ☐ b. Does the system have at least two actuators?
- ☐ c. Does the system have at least two sensors?
- ☐ d. Does it function correctly with no Bluetooth central connected?
- ☐ e. Can it be connected to?
- ☐ f. Does the bluetooth interface function as expected?

Finally! Before you turn in your lab submission, please take a few minutes to reflect on the process of designing and building this final project as a member of a team.

Deliverable 7: Reflection (*3 pts*)

In your own words, describe what you found challenging about this project. Describe the division of labor your team used to complete this task. Explain clearly your contributions to the final design. Use the space below or attach additional pages to reflect on this process.